



Computer Science and Engineering Program

**ADAPTIVE HONEYPOT ARCHITECTURES USING INTELLIGENT
DIGITAL SYSTEMS FOR CYBER THREAT DETECTION**

Bachelor Degree

Eron Mustafa

June / 2026
Prishtinë



Computer Science and Engineering Program

Bachelor Thesis
Academic Year 2020-2021

Eron Mustafa

**ADAPTIVE HONEYPOT ARCHITECTURES USING INTELLIGENT
DIGITAL SYSTEMS FOR CYBER THREAT DETECTION**

Supervisor: Prof. Dr. Blerton Abazi

June / 2026

This thesis was compiled and submitted in partial fulfillment of requirements
for the Bachelor Degree

ABSTRACT

Honeypots are security resources that are specifically made to be probed, attacked, and compromised in order to provide valuable information about the actions of attackers and new threats. This thesis explores the design implementation and assessment of an adaptive honeypot architecture using a functional prototype called HoneypotAI. The prototype includes a public decoy portal, a rule-based detection pipeline, optional machine learning enrichment, MongoDB-based persistence, and an authenticated operator dashboard for real-time monitoring. The system is built with Node.js and Express, which host both the public honeypot routes and the protected dashboard API on separate paths. The public decoy surface and the protected dashboard surface are separated by a JS and Express application. When a database connection is available incoming requests are stored in MongoDB after being logged categorized using heuristic detection rules and potentially enhanced by an external machine learning service. An operator is then presented with summary statistics charts map points and event details by the dashboard. The prototypes support for realistic traffic capture adaptive responses like delay tarpit fake data and fake routes as well as a useful workflow for event review are all demonstrated in the implementation report. Instead of being presented as a large-scale production experiment the results are presented as an evaluation of a prototype at the bachelor level.

ACKNOWLEDGEMENTS

I would like to express my deep gratitude to the respected Prof. Dr. Blerton Abazi for his tireless work and support, not only in the finalization of this thesis, but also for his outstanding teaching. I had the special pleasure of knowing him and being his student.

Last but not least, I am pleased to say that I dedicate this thesis to my family, the endless source of love, motivation, and goodwill, who were with me in every step of this very important journey.

CONTENTS

LIST OF FIGURES	1
GLOSSARY OF TERMS	2
1. INTRODUCTION	4
2. LITERATURE REVIEW	5
2.1 Honeypot Fundamentals	5
2.2 Intelligent Threat Detection Systems	6
2.3 Theoretical Comparison of Detection Approaches	7
2.3.1 Detection Rate and False Positives	7
2.3.2 Scalability and Maintainability	7
2.3.3 Security and Complexity Management	8
2.4 Honeypot Architectures in Different Contexts	9
2.5 Adaptive and Hybrid Approaches	10
3. PROBLEM STATEMENT	12
4. METHODOLOGY	14
4.1 Research Methodology	14
4.2 System Architecture Design	14
4.3 Machine Learning Model Training	16
4.4 Experimental Setup	17
4.5 Prototype Implementation Report	17
4.6 Request and Data Flow	18
4.7 Dashboard and Security Model	20
4.8 Operational Deployment and Reset Workflow	21
4.9 Test Environment and Steps	22
5. RESULTS	23
5.1 Detection Performance Results	23
5.2 Adaptive Response Evaluation	23
5.3 Network Traffic Analysis	24
5.4 Study Limitations	26
5.5 Prototype Demonstration Report	26
5.6 Dashboard and Deployment Observations	27

5.6.1 Admin Dashboard During Tests	28
5.7 Production Server Log Sample	28
6. DISCUSSION AND CONCLUSIONS	30
7. REFERENCES	31

LIST OF FIGURES

Figure 1. Honeypot classification taxonomy – Source: IIETA	5
Figure 2. Adaptive Honeypot Architecture Diagram	15
Figure 3. Machine Learning Pipeline for Threat Classification	16
Figure 4. Public Decoy Portal Route and Dashboard Separation	19
Figure 5. Dashboard Login and Protected Operator Console.....	21
Figure 6. Traffic Analysis.....	25

GLOSSARY OF TERMS

API - Application Programming Interface

AUC - Area Under the Curve (ROC)

BoT - Bot (automated client or script)

Certbot - Tool used to obtain and renew TLS certificates from Let's Encrypt

CSS - Cascading Style Sheets

HTML - HyperText Markup Language

HTTP - HyperText Transfer Protocol

HTTPS - HyperText Transfer Protocol Secure

IP - Internet Protocol

IT - Information Technology

IoT - Internet of Things

JSON - JavaScript Object Notation

JWT - JSON Web Token

ML - Machine Learning

MongoDB - Document-oriented NoSQL database used for attack logs and profiles

npm - Node Package Manager

OS - Operating System

PC - Personal Computer

pm2 - Process Manager 2 (Node.js process manager used on the VPS)

REST - Representational State Transfer

SQL - Structured Query Language

TCP - Transmission Control Protocol

TLS - Transport Layer Security

URI - Uniform Resource Identifier

URL - Uniform Resource Locator

VPS - Virtual Private Server

XSS - Cross-Site Scripting

1. INTRODUCTION

In recent times, there has been a growing interest in security and information protection for network systems. Web applications store valuable data that must be protected from attackers, yet many threats today are automated which they risk a lot. Nowadays, researchers and security experts often use honeypots to monitor activity, learn attack techniques, and improve threat detection without exposing production systems. According to Spitzner (2002), founder of the HoneyNet Project, “a honeypot is a security resource whose value lies in being probed, attacked, or compromised” [1]. In other words, a honeypot is a decoy placed on a network as bait to lure attackers. It is designed to look like a real system, with open services and attractive targets, so that malicious interaction can be recorded and analyzed for research and security purposes.

The thesis does not stay only on theory. It also walks through HoneyPotAI step by step: how the fake site was set up, how requests are logged, how attacks are spotted, and how results appear on the admin dashboard. Research ideas and project work are written together so it is clear what was read and what was actually built.

HoneyPotAI is an adaptive web honeypot aimed at web-based threats. The backend runs on Node.js and Express. Visitors see a normal-looking portal with login forms, admin paths, search, and URLs that resemble config or backup files. Behind that surface, rules check traffic for SQL injection, XSS, scanning, brute-force logins, and similar patterns. The system keeps a record per IP and adjusts replies when behavior looks more aggressive, sometimes with slower responses or fake data meant to keep the session going.

Attack data goes to MongoDB when the database is available; Otherwise, the app can run in memory for local tests. The admin dashboard shows counts and charts from stored events. Machine learning is optional: a small Python service can rank behavior from request patterns if it is enabled in the configuration.

2. LITERATURE REVIEW

2.1 Honeypot Fundamentals

The difference between a honeypot and a production system is that a production system provides real services to legitimate users, while a honeypot is intentionally created as a decoy. Cohen explains that honeypots and decoys are used to “cause malicious actors to attack the wrong targets” and to “produce potentially useful information for defenders” [3]. Based on this, honeypots are useful because they allow suspicious traffic and attacker behavior to be observed without exposing the real system.

Honeypots can also be discussed based on their interaction level. Provos and Holz describe high-interaction honeypots as systems that provide “Real services, OS, or applications,” while low-interaction honeypots are based on “Emulation of TCP/IP stack, vulnerabilities” [2]. This means that high-interaction honeypots allow deeper attacker interaction and can “Capture extensive amount of information,” but they also have “Higher risk” [2]. Low-interaction honeypots, on the other hand, have “Lower risk” and mainly “Capture quantitative information about attacks” [2].

Despite their usefulness, honeypots can lose value if attackers are able to detect them. Bringer et al. state that “the true value in honeypots is in monitoring attackers’ activities without being detected” [4].

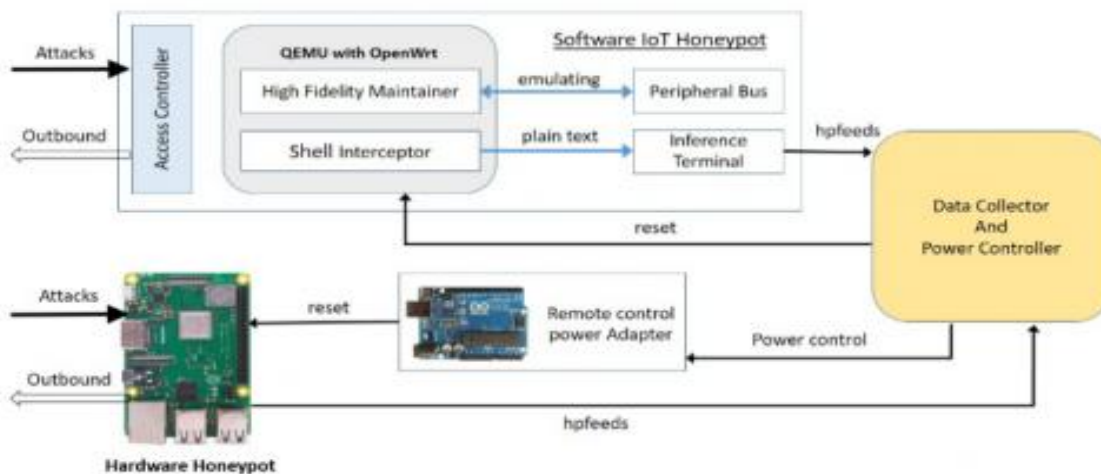


FIGURE 1. HONEYPOT CLASSIFICATION TAXONOMY – SOURCE: [IIETA](#)

2.2 Intelligent Threat Detection Systems

Machine learning is often discussed in intrusion detection through two main approaches: misuse detection and anomaly detection. Sommer and Paxson describe misuse-detection systems as systems that use “precise descriptions of known malicious behavior,” while anomaly-detection systems “flag deviations from that profile” [5]. This distinction is relevant for HoneyPotAI because the platform uses rule-based checks for known web attack patterns, while the optional machine learning service can support behavior classification from request patterns.

At the same time, machine learning cannot be treated as a perfect solution for intrusion detection. Sommer and Paxson mention problems such as “lack of training data” and the “very high cost of errors” [5]. For this reason, the machine learning part of HoneyPotAI is not used as the only detection method. It is added as a supporting component, while the main system still relies on direct request analysis, attack rules, logging, and attacker profiling.

Ferrag et al. proposed RDTIDS, an intrusion detection system that “combines different classifier approaches” using decision-tree and rule-based methods [6]. Their system was tested on the “CICIDS2017 dataset and BoT-IoT dataset” [6]. The results reported in their work show an accuracy of 96.665% for CICIDS2017 and 96.995% for BoT-IoT [6]. This shows that classifier-based approaches can perform well on known datasets, although the results still depend on the quality and relevance of the training data.

Anomaly detection is also important because not every attack has the same fixed signature. Chalapathy and Chawla present a survey of deep learning methods for anomaly detection and describe it as a field that studies how to “differentiate between normal and anomalous behavior” [7]. In the context of honeypots, this is useful because suspicious behavior may appear as repeated requests, unusual paths, strange payloads, or a sequence of actions from the same source IP address.

Deep learning has also been used for intrusion detection. Yin et al. proposed an intrusion detection model based on “recurrent neural networks (RNN-IDS)” and evaluated it for “binary classification and multiclass classification” [8]. Their work is relevant because it

shows that traffic or attack behavior can be classified into different categories. In HoneyPotAI, this idea is applied in a simpler way through the optional Python machine learning service, which classifies attacker behavior based on request patterns.

2.3 Theoretical Comparison of Detection Approaches

2.3.1 Detection Rate and False Positives

Static honeypots can be useful for collecting information about common attacks, because honeypots act as “electronic baits” that lure attackers and help assess vulnerabilities [9]. Holz and Raynal also explain that “Every interaction with the system is suspicious,” which gives honeypots an advantage over traditional intrusion detection systems [9]. However, the same authors show that attackers can detect honeypots and suspicious environments, and they state that “current malware already implements techniques to do so” [9]. This means that static honeypots may be less effective when attackers or malware are able to recognize that they are interacting with a decoy.

Adaptive honeypots address this limitation by changing their behavior based on attacker interaction. Dowling et al. describe an adaptive honeypot that uses reinforcement learning and “learns the best responses to overcome initial detection attempts” [10]. Their system is designed for “prolonging attack interaction” and was able to collect “four times more attack command transitions” than a standard high-interaction honeypot [10]. This supports the idea that adaptive honeypots can collect a larger and more useful dataset by making it harder for malware to detect the honeypot and stop the attack.

2.3.2 Scalability and Maintainability

In this project, scalability is related to how easily the honeypot can be repeated, changed, and extended without rebuilding the whole system each time. This is one of the reasons why container-based honeypot deployment is useful. Instead of treating every honeypot component as one large system, the services can be separated and managed more independently. Khorshed et al. discuss this idea in relation to container-based honeypot deployment for dynamic threat intelligence, where the deployment approach supports the collection and management of attack information [11].

This idea is also visible in T-Pot. The platform is described as an “all in one” multi-honeypot platform that supports “20+ honeypots” and includes visualization options through the Elastic Stack, live attack maps, and different security tools [18]. For this reason, T-Pot is a useful example of how several honeypot services can work in the same environment instead of being installed and maintained as separate systems. This is relevant to HoneyPotAI because the platform also combines different parts, such as decoy routes, attack detection, logging, attacker profiling, and dashboard visualization.

Maintainability was also considered during the implementation. The system was designed so that new suspicious paths, detection rules, and response behaviors can be added without changing the full structure of the application. The adaptive behavior engine supports this by allowing the honeypot to respond differently depending on the observed attacker activity. For example, repeated login attempts, suspicious payloads, or requests to sensitive-looking paths can lead to different responses. In this way, the platform remains easier to update than a completely static honeypot.

2.3.3 Security and Complexity Management

A high-interaction honeypot can provide useful information, but it also introduces risk if the attacker is allowed too much freedom inside the environment. The HoneyNet Project explains this problem through the concept of data control. According to Know Your Enemy, “Data Control is the containment of activity” [12]. This means that the honeypot must be controlled so that activity inside it does not affect systems outside the honeypot environment.

The same source also explains that, after an attacker enters the honeynet or a system is compromised, the goal is that “they cannot accidentally or purposefully harm other non-honeynet systems” [12]. For this reason, containment is an important part of honeypot security. This idea is applied in HoneyPotAI by avoiding access to real production data and using decoy routes, fake responses, logging, and attacker profiling to observe attacker behavior without exposing real systems.

Security management becomes more difficult when the honeypot has more interaction points. Every new route, fake service, or response behavior must be controlled so that it collects

useful information without creating unnecessary risk. In this project, this was handled by keeping the honeypot interaction limited to the web application layer and by storing attacker activity through logs and profiles. This keeps the platform useful for analysis while still keeping the environment separated from real services.

2.4 Honeypot Architectures in Different Contexts

Honeypot architecture depends on the purpose of the deployment and the type of activity that needs to be observed. The HoneyNet Project explains that the “primary purpose of a honeynet is to gather information on threats” [12]. Since a honeynet is not a normal production system, “any interaction with a honeynet implies malicious or unauthorized activity” [12]. For this reason, honeynets are useful in environments where the goal is to observe suspicious behavior and collect security data rather than provide real services.

Another important context is honeypot data analysis and attribution. Fraunholz et al. focus on the attribution of honeypot data, which shows that collected events need to be analyzed before they become useful intelligence [13]. In practice, honeypot logs are not only stored as raw requests. They also need to be connected to attacker behavior, source IP addresses, payloads, repeated actions, and other indicators that help explain what the attacker was trying to do.

IoT honeypots require a different architecture from web or enterprise honeypots. Pa Pa et al. show that “Telnet-based attacks that target IoT devices have rocketed since 2014” [14]. They proposed IoTPOT, an IoT honeypot and sandbox that attracts and analyses Telnet-based attacks against “various IoT devices running on different CPU architectures such as ARM, MIPS, and PPC” [14]. This shows that IoT honeypots need to imitate device behavior and CPU environments, while HoneyPotAI focuses on web application behavior.

Large-scale research systems are another deployment context. CAIDA describes the UCSD Network Telescope as a “passive traffic monitoring system” built on a “globally routed, but lightly utilized /9 and /10 network” [15]. After filtering legitimate traffic, CAIDA explains that the remaining traffic gives a continuous view of anomalous unsolicited traffic [15]. This

type of architecture is useful for studying broad Internet activity such as scanning, worms, and denial-of-service backscatter.

HoneypotAI follows a narrower web application architecture. It does not try to emulate a full enterprise network, IoT device, or Internet telescope. Instead, it provides fake web routes, login pages, admin areas, search endpoints, and sensitive-looking paths. The purpose is to observe attacks against web applications, store them as logs and attacker profiles, and show them through a dashboard.

2.5 Adaptive and Hybrid Approaches

Adaptive and hybrid honeypot approaches are useful because attackers and attack techniques do not always follow fixed patterns. Bilge and Dumitras show this problem in their study of zero-day attacks, where they identified “18 vulnerabilities exploited before disclosure” and found that “a typical zero-day attack lasts 312 days on average” [16]. This supports the need for systems that collect behavior and attack evidence, not only systems that depend on already known signatures.

The value of a honeypot still depends on attacker interaction. Spitzner defines a honeypot as “a security resource whose value lies in being probed, attacked, or compromised” [1]. The Honeynet Project also states that the “primary purpose of a honeynet is to gather information on threats” [12]. In this thesis, the adaptive part of HoneypotAI follows this idea. The system changes some responses based on the activity it observes, such as scanning, repeated login attempts, suspicious payloads, or requests to sensitive-looking paths.

A related concept is moving target defense. Zhu and Basar describe it as a proactive defense scheme that “dynamically alters the attack surface of the network” and makes it more difficult for attackers to gather system information [17]. HoneypotAI does not implement a full moving target defense model, but it uses a similar idea at the web application level. Instead of giving the same response to every request, the honeypot can adapt the response according to the attacker profile and detected behavior.

Hybrid honeypot systems combine different sensors, services, and analysis tools in one environment. T-Pot is described as an “all in one” multi-honeypot platform that supports

“20+ honeypots” and includes visualization options, live attack maps, and security tools [18]. HoneypotAI applies this hybrid idea on a smaller scale by combining decoy web routes, attack detection, attacker profiling, MongoDB logging, dashboard visualization, and a machine learning service in one platform.

3. PROBLEM STATEMENT

Although honeypots are useful for observing attacker behavior, their practical use still has some limitations. A honeypot only has value if attackers interact with it and do not immediately recognize that it is a decoy. Bringer et al. explain that “the true value in honeypots is in monitoring attackers’ activities without being detected” [4]. This creates a problem for static honeypots, because attackers can try to identify artificial environments before continuing their activity. Holz and Raynal also discuss the detection of honeypots and suspicious environments, showing that honeypot fingerprinting is a real concern in this field [9].

Another problem is that a honeypot must be controlled and maintained carefully. If a honeypot allows too much interaction, there is a risk that an attacker may try to use it for actions outside the intended environment. The HoneyNet Project describes this through the concept of data control, which is defined as “the containment of activity” [12]. This means that the honeypot must collect useful information, but it must also remain separated from real systems and production data.

A further issue is that security teams need useful information from the collected activity, not only raw logs. Mandiant defines dwell time as the time between compromise and discovery, and M-Trends 2022 reported a global median dwell time of 21 days [19]. This shows that attackers may remain active for some time before they are detected. For this reason, honeypot data should be logged, structured, and analyzed in a way that helps identify suspicious behavior more quickly.

This thesis addresses these issues through the design and implementation of HoneyPotAI, an adaptive web honeypot platform. The system focuses on web application attacks and provides decoy routes such as login pages, admin panels, search endpoints, and sensitive-looking paths. It also includes attack detection, attacker profiling, MongoDB logging, dashboard visualization, and a separate machine learning service. The main objectives of this thesis are:

- To design a modular web honeypot architecture that can attract and record suspicious web traffic without exposing real production data.

- To implement detection rules for common web attacks, including SQL injection, cross-site scripting, command injection, scanning, and brute-force login attempts.
- To develop an attacker profiling component that tracks source IP addresses over time, classifies behavior, and assigns a threat level.
- To implement an adaptive behavior engine that changes some honeypot responses based on observed attacker activity.
- To develop a dashboard that presents attack logs, statistics, and attacker information in a clearer form for analysis.
- To include a machine learning service that can classify attacker behavior based on request patterns.

The expected outcome of this thesis is a working prototype of an adaptive web honeypot platform. The goal is not to replace existing security systems, but to show how honeypot data can be collected, organized, and used to better understand web-based attack behavior.

4. METHODOLOGY

4.1 Research Methodology

This thesis follows a design science research methodology because the main work of the thesis is the design and implementation of a working system. Hevner et al. describe design science as research that “creates and evaluates IT artifacts” to solve identified problems [20]. In this thesis, the artifact is HoneypotAI, an adaptive web honeypot platform developed to detect and study web-based attacks.

The work started with a review of literature related to honeypots, honeynets, machine learning for intrusion detection, adaptive deception, and honeypot data analysis. This helped define the main features that the system should include, such as decoy web routes, attack detection, attacker profiling, adaptive responses, logging, and dashboard visualization.

After the requirements were defined, the prototype was implemented. The backend was built with Node.js and Express, while MongoDB was used to store attack logs and attacker profiles. I also added an in-memory fallback so that the system could still be tested locally when the database was not available. The public honeypot surface included fake login pages, admin routes, search endpoints, and sensitive-looking paths. The last part of the methodology was testing the prototype.

The system was tested using simulated attacks such as SQL injection, cross-site scripting, command injection, scanning, and brute-force login attempts. During testing, I checked whether the attacks were detected correctly, whether attacker profiles were updated, and whether the dashboard showed the collected information in a clear way. The machine learning service was also tested as an additional component for classifying attacker behavior from request patterns.

4.2 System Architecture Design

HoneypotAI is implemented with Node.js and Express as a single web application. The design separates what is public from what is private. The decoy portal stays open so scanners

and bots can reach it, while the dashboard is restricted and used only to inspect captured traffic.

When someone requests a decoy page (for example /login, /admin, or /search), the request is handled in a fixed order. Middleware records the IP, path, method, and headers. The detection module then checks the content for signs of SQL injection, XSS, directory scanning, or repeated login attempts. Results from this step are passed to the profile manager, which maintains one record per IP and updates classification and threat score as activity continues. The adaptive engine reads that profile and chooses how to answer. In some cases the reply is delayed; in others the system returns fake database rows or extra decoy paths so the session lasts longer.

Attack logs and profiles are stored in MongoDB. If MongoDB is not connected, the same data is kept in memory for local runs. Geolocation is added when enrichment is enabled. The operator console is a simple web UI that calls protected endpoints under /dashboard/api/; Chart.js and Leaflet are used for charts and the origin map. A Python ML service is optional and is contacted only when machine learning is turned on in configuration.

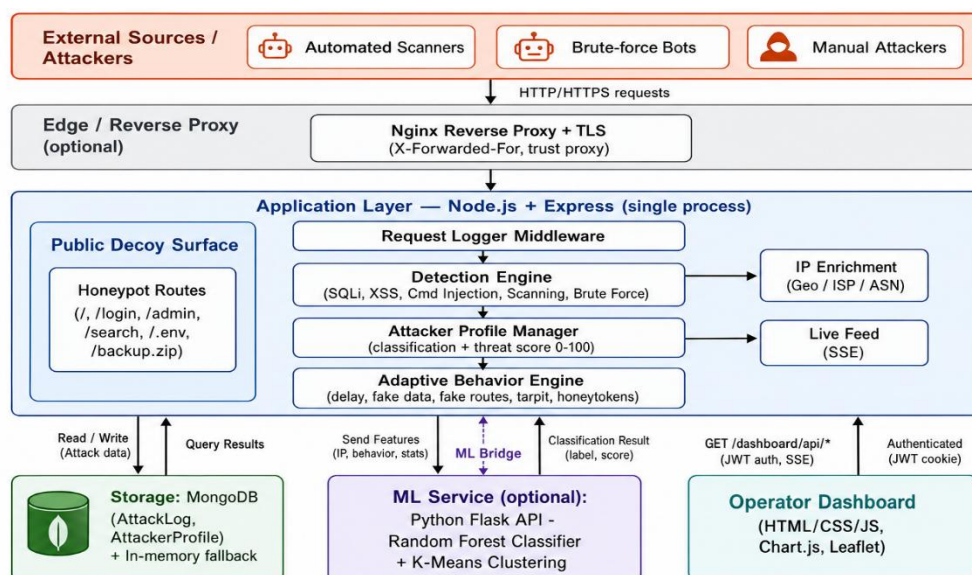


FIGURE 2. ADAPTIVE HONEYPOT ARCHITECTURE DIAGRAM

4.3 Machine Learning Model Training

Machine learning in HoneypotAI is optional and runs as a separate Python service. It supports the main detection logic but does not replace it.

The system uses a Random Forest classifier from scikit-learn. The model classifies attacker profiles into seven groups: normal, scanner, brute_forcer, sql_attacker, xss_attacker, cmd_attacker, and advanced. Thirteen behavioral features are used, such as total requests, malicious requests, login attempts, attack type counts, request frequency, and session duration. These features are taken from attacker profiles in the Node.js backend and sent to the Python service when machine learning is enabled.

Training was done with synthetic data generated in the project, not with external datasets like CICIDS2017 or UNSW-NB15. The data was split into training and test sets, features were scaled, and the model was trained and then checked using accuracy and a classification report. The trained model is saved and loaded by the Flask API.

K-Means clustering is also included to group similar attacker profiles and help identify common behaviour patterns. The Python service exposes REST endpoints for classification and clustering, and the Node.js application connects to it through an internal bridge when ML_ENABLED is set to true.

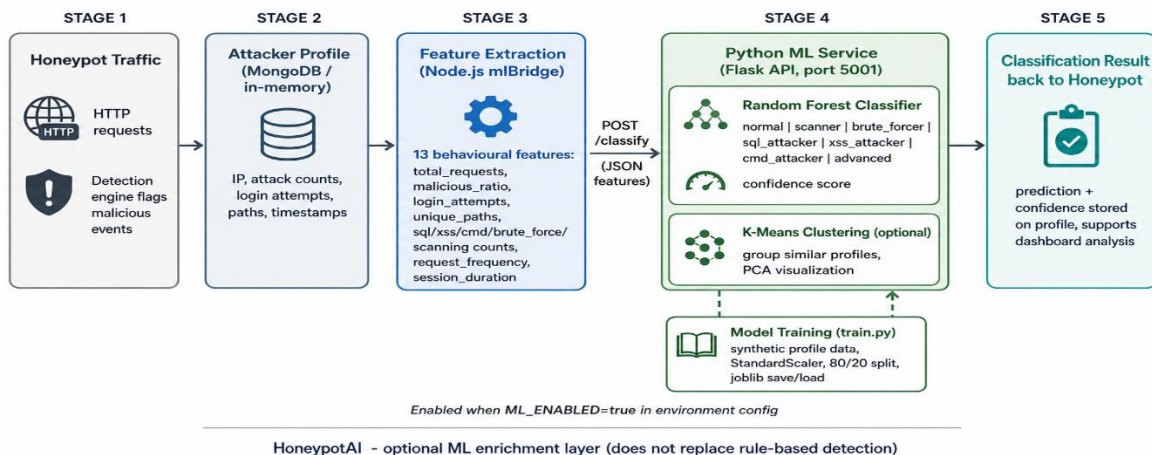


FIGURE 3. MACHINE LEARNING PIPELINE FOR THREAT CLASSIFICATION

4.4 Experimental Setup

All tests for this project were run on a local machine. By default, the simulators targeted `http://127.0.0.1:3000`, but this could be changed in the `.env` file if another host or port was used.

To create attack traffic, I did not use external tools like Metasploit or Nmap. Instead, I relied on three scripts already included in the repository. One script repeated login attempts against `/login` to simulate brute force. Another sent SQL injection payloads to `/search`, `/login`, and `/api/query`. The third one requested many common paths such as `/admin`, `/.env`, and `/backup.zip` to imitate scanner behavior. I ran them with `npm run test:bruteforce`, `npm run test:sqli`, and `npm run test:scanner`, and sometimes with `npm run test:all` when I wanted to run everything in sequence.

When a simulation finished, I opened the dashboard and checked whether the traffic appeared in the logs. I looked at attack type detection, IP profiling, adaptive responses, and the live feed. If machine learning was enabled, I also checked whether the Python service returned a classification result. The goal of these tests was to confirm that the full system worked from incoming request to stored log and dashboard view.

4.5 Prototype Implementation Report

The prototype developed in this project is called HoneypotAI. I built it as a web-based adaptive honeypot so the thesis would include a working system, not only a design on paper. The application shows a fake public website to attackers, processes each request through detection and profiling logic, stores the results, and displays them in a protected dashboard.

The backend is written in Node.js with Express. The entry point is `server/server.js`, which starts the application and checks important configuration values before runtime. The main setup is in `server/app.js`, where middleware, public routes, dashboard routes, authentication, and static files are connected. Public decoy routes are kept in `server/routes/honeypot.js`, while operator routes and API endpoints are in `server/routes/dashboard.js`. I separated these on purpose because the honeypot side must stay open and realistic, while the dashboard must remain private.

Most of the security logic is placed in server/services. This includes detection, attacker profiling, adaptive responses, IP enrichment, and the optional machine learning bridge. Data is stored using Mongoose models such as AttackLog for individual events and AttackerProfile for repeated behavior from the same IP. If MongoDB is unavailable, the system can still run in memory mode for local testing.

The dashboard is a simple frontend in the dashboard folder, built with HTML, CSS, and JavaScript. It uses Chart.js for charts and Leaflet for the map. The browser does not access the database directly. It requests prepared data from /dashboard/api/* after login. For this prototype, I avoided a large frontend framework because the main goal was to monitor attacks clearly, not to build a complex client application.

4.6 Request and Data Flow

When a visitor or bot hits the public decoy, the request is handled like a normal web call on the outside, but inside the application it is treated as something to record and analyze. The middleware captures the IP, method, path, headers, user agent, and timing before the response is sent. This matters because one request alone often says little, but a series of paths such as /admin, /.env, and /login usually points to scanning or probing.

After logging, the detection engine checks the request with rule-based heuristics for SQL injection, XSS, command injection, scanning, and brute-force behavior. The profile manager then updates a record for that IP and adjusts its classification and threat score. If machine learning is enabled, features from the profile can be sent to the Python service for an extra classification result. The adaptive engine uses the profile to decide how to reply. In some cases, it adds a delay, returns fake data, or exposes another decoy route instead of giving the same static page every time.

When the response finishes, the event is stored as an AttackLog entry. Repeated activity from the same source is also reflected in AttackerProfile. If MongoDB is not connected, the same information is kept in memory for local runs. The dashboard does not read the database directly. It loads prepared summaries and lists from /dashboard/api/*, while the live feed shows new events as they arrive. IP enrichment is added when available, mainly to support

map views and basic source context such as country, city, and network provider information.

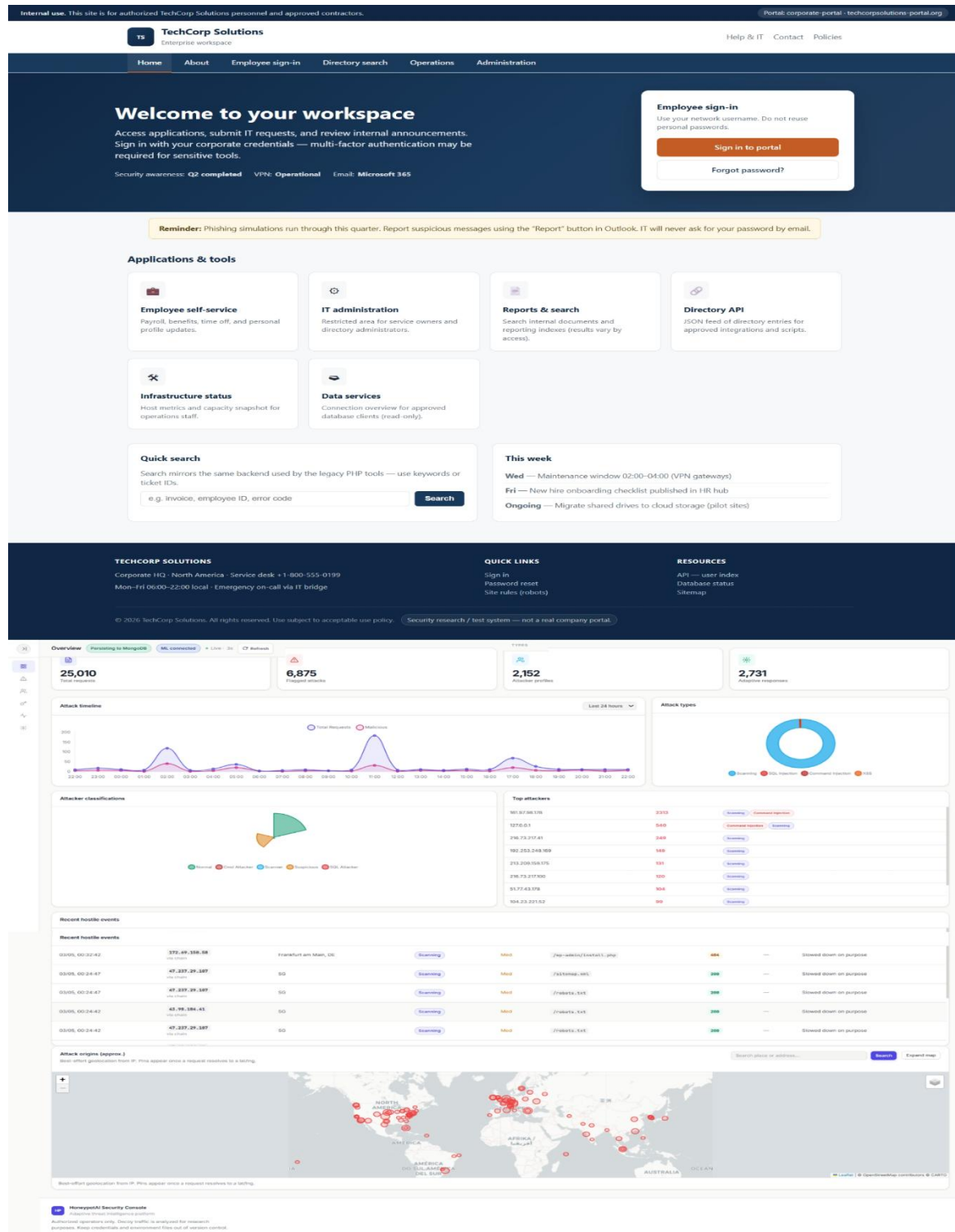


FIGURE 4. PUBLIC DECOY PORTAL ROUTE AND DASHBOARD SEPARATION

4.7 Dashboard and Security Model

The dashboard is the admin side of HoneypotAI. It runs on `/dashboard`, while the fake portal stays on `/`. Attackers can browse the decoy paths, but only the admin should open the console where logs, charts, and profile data are shown.

The UI was kept simple on purpose: HTML, CSS, and JavaScript only. `dashboard.js` loads the pages and calls the backend; `charts.js` builds the Chart.js graphs; Leaflet draws the map from geo points returned by the API. Stats and counters update every few seconds through `/dashboard/api/stats`. For fresh events the admin can watch `/dashboard/api/live`, which uses Server-Sent Events instead of reloading the whole page. Logs, profiles, credentials, and adaptive stats all come from routes under `/dashboard/api/`.

Nothing in the browser reads MongoDB directly. The server prepares the JSON first, mainly in `server/routes/dashboard.js`, so the frontend only renders what it receives. When old test data needs to be removed, the admin sends `POST /dashboard/api/reset-data` with JSON, the header `X-Dashboard-Intent: reset-operator-data`, and `"confirm": true`. That clears Mongo collections when connected, deletes log files if possible, and resets the in-memory counters used by detection and adaptive modules.

Login happens at `/dashboard/login`. After valid admin credentials, the app stores an HS256 JWT in the `honeypot_dash_token` cookie. The cookie is `httpOnly`, uses `sameSite=lax`, and is limited to `/dashboard`, so it should not travel to honeypot URLs like `/login` or `/admin`. Sessions expire after idle time set by `DASHBOARD_IDLE_MINUTES` and refresh while the admin stays active. Brute forcing the login form is slowed by rate limiting, and `DASHBOARD_ALLOWED_IPS` can block unknown IPs on a public server. Production

startup also rejects weak dashboard passwords and signing keys before the app is allowed to run.

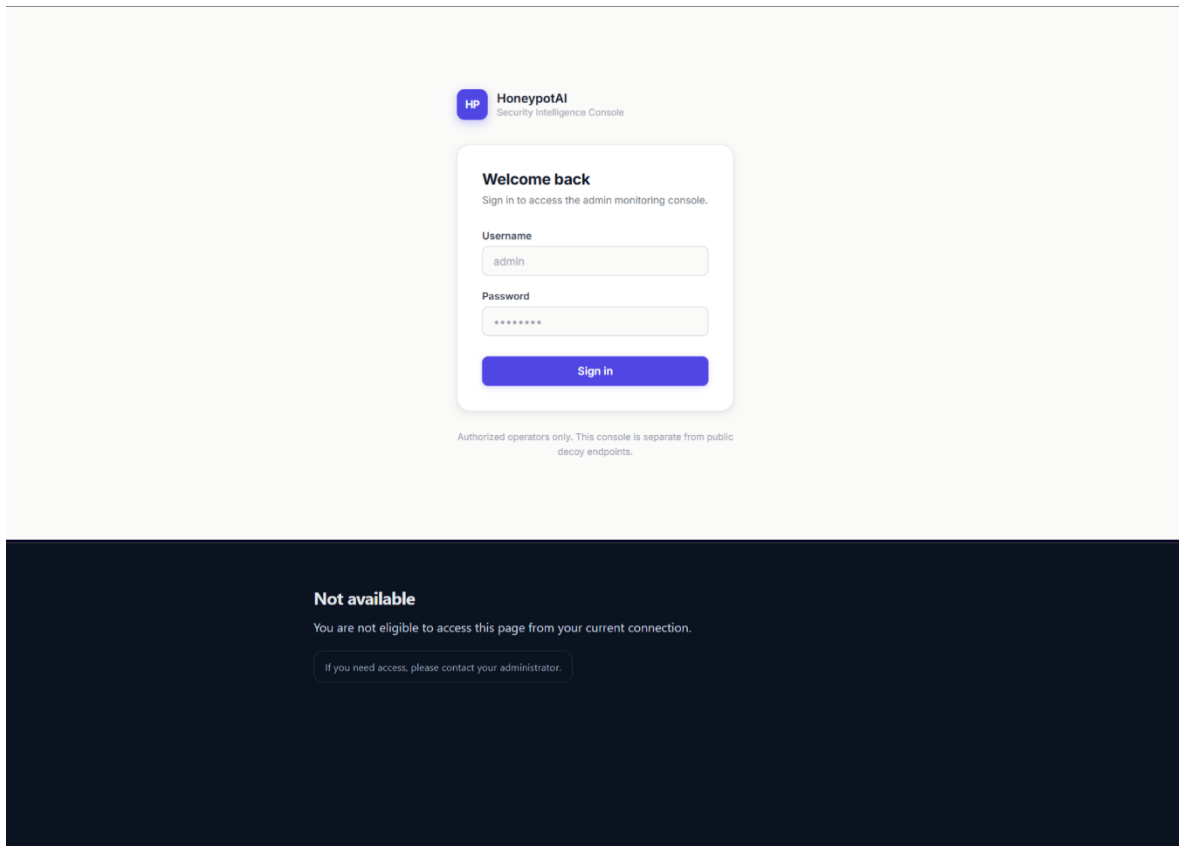


FIGURE 5. DASHBOARD LOGIN AND PROTECTED OPERATOR CONSOLE

4.8 Operational Deployment and Reset Workflow

Local setup is kept basic. After npm install, the app starts with npm run dev or npm start. The terminal prints the honeypot URL and dashboard URL from the current port settings, so the admin can open the decoy in one tab and the console in another. The optional ML service is turned on with ML_ENABLED=true and ML_SERVICE_URL pointing to the Python API, usually http://127.0.0.1:5001. Using 127.0.0.1 instead of localhost avoids IPv4/IPv6 mismatch during testing.

On a VPS, Node should run under pm2 or systemd so it survives reboots. Nginx can sit in front as a reverse proxy and handle TLS, often with Certbot. MongoDB needs a

real MONGODB_URI, for example Atlas or a secured local instance, because attack logs and profiles should survive restarts. Without it the app still works in memory mode, but that data disappears when the process stops.

Behind a proxy, TRUST_PROXY_HOPS must match the setup so req.ip and X-Forwarded-For resolve to the real client. Wrong IP handling would break profiling and map views in the dashboard.

For repeated test runs, data can be cleared in two ways. The script `npm run reset:dashboard` removes log files under `logs/` and wipes `AttackLog` and `AttackerProfile` in MongoDB when configured. While the server is running, the admin can also call `POST /dashboard/api/reset-data` from the dashboard, which clears database rows, log files, and in-memory counters. A clean reset helps when collecting screenshots or thesis results, so charts and live feed numbers match only the current simulation and not older traffic mixed in.

4.9 Test Environment and Steps

HoneypotAI was built and tested on a normal PC with Node.js. MongoDB was used when logs had to survive a restart. Settings such as ports, database URI, dashboard login, and ML flags were taken from the `.env` file.

Before a clean run, old data was cleared with `npm run reset:dashboard` or the admin reset button. Then the server was started with `npm run dev`. One simulator was run at a time: `npm run test:bruteforce` sends 20 login POSTs to `/login`; `npm run test:sqli` sends strings to `/search`, `/login`, and `/api/query`; `npm run test:scanner` GETs many paths like `/admin`, `/.env`, and `/backup.zip`.

Brute-force tagging starts after five login attempts from one IP. Scanner tagging starts after ten different paths. SQL attacker tagging starts after three SQLi detections. These numbers come from env vars and can be changed without editing code.

After each script, the admin logged into `/dashboard` and checked Attacks for new rows, Profiles for the same IP, and the Live feed for fresh events. Screenshots were saved immediately because the research server deletes logs on a schedule and does not keep months of history.

5. RESULTS

5.1 Detection Performance Results

This chapter reports what happened when HoneypotAI was tested locally. The work did not use a public attack dataset or a formal benchmark, so there are no accuracy, AUC, or large-scale detection rates to quote. The tests were mainly about seeing whether attack traffic reached the decoy, got picked up by the detection layer, landed in storage, and showed up in the admin dashboard.

Three small simulators from the repo generated the traffic. One kept posting login attempts to /login. Another sent SQL injection strings to /search, /login, and /api/query. The third walked through paths like /admin, /.env, and /backup.zip the way a scanner might. When these scripts finished, the attack log listed the events with types such as brute force, SQL injection, and scanning whenever the rules matched.

Overall, the detection side worked as intended. A normal visit to the home page stayed in the background, but repeated login tries or a burst of different paths from the same IP raised the threat level and updated the profile. During the scanner run, the source moved toward a scanner label after enough unique paths were hit. SQL tests were flagged as malicious, and some replies came back with fake database rows rather than a simple error, which showed the adaptive layer reacting to that traffic.

The Python ML module was left out of formal scoring here. It can still be turned on with `ML_ENABLED` and `ML_SERVICE_URL` if the Flask service is up, but the runs above relied on the rule engine and profile manager. That was sufficient for this thesis to show the chain from request to log entry to dashboard screen.

5.2 Adaptive Response Evaluation

Adaptive behavior was checked during the same local tests used for detection. After the simulators sent traffic to the decoy, the admin dashboard and terminal output were used to see whether replies changed based on attacker type.

The honeypot does not always return the same response. For repeated login attempts on /login, responses became slower and the source profile moved toward brute-force

behavior. SQL injection tests on /search sometimes returned fake database-style content instead of a simple error page. When the scanner script requested many paths from one IP, the system classified the activity as scanning and the adaptive layer responded in line with that label.

These results were checked by observation, not by formal timing measurements. Dwell time, delay values, and comparison with a static honeypot were not calculated in this project. Even so, the tests showed that adaptive actions are connected to detection and profiling, and they appeared when attack patterns increased during simulation.

The brute-force script was the easiest case to notice in the terminal. Later /login attempts sometimes took longer to answer, and the profile showed more login tries. On /search, a few SQL payloads returned JSON that looked like table rows instead of a normal error page. The scanner script printed many 200 responses for decoy paths; in the dashboard that same IP soon had a high path count. None of this was timed with a stopwatch. The check was simply: run script, watch dashboard, read a few responses.

5.3 Network Traffic Analysis

Network traffic in this thesis means the requests generated during local prototype testing, not live internet traffic collected over weeks or millions of connections. The point was to see whether HoneypotAI could record each request with enough detail for later review in the admin dashboard.

The simulators and manual checks produced two kinds of traffic. Some requests looked like normal page views on / or other public pages. Others targeted admin-looking routes, login forms, and paths that scanners often try, such as /.env and /backup.zip. For each event the system stored the method, path, source IP, headers, timing, and whatever the detection layer attached. That data fed the counters, charts, and live feed.

With MongoDB connected, events went into AttackLog and repeated activity from one IP built up in AttackerProfile. Without MongoDB the app still ran, but stats lived in memory only and were lost after restart, so MongoDB was used when screenshots and report material

needed to stay consistent. The admin dashboard pulled summaries from the backend rather than raw database rows. Map points depended on IP enrichment; when coordinates were missing, those events simply did not appear on the map, which is expected for localhost or incomplete lookup data.

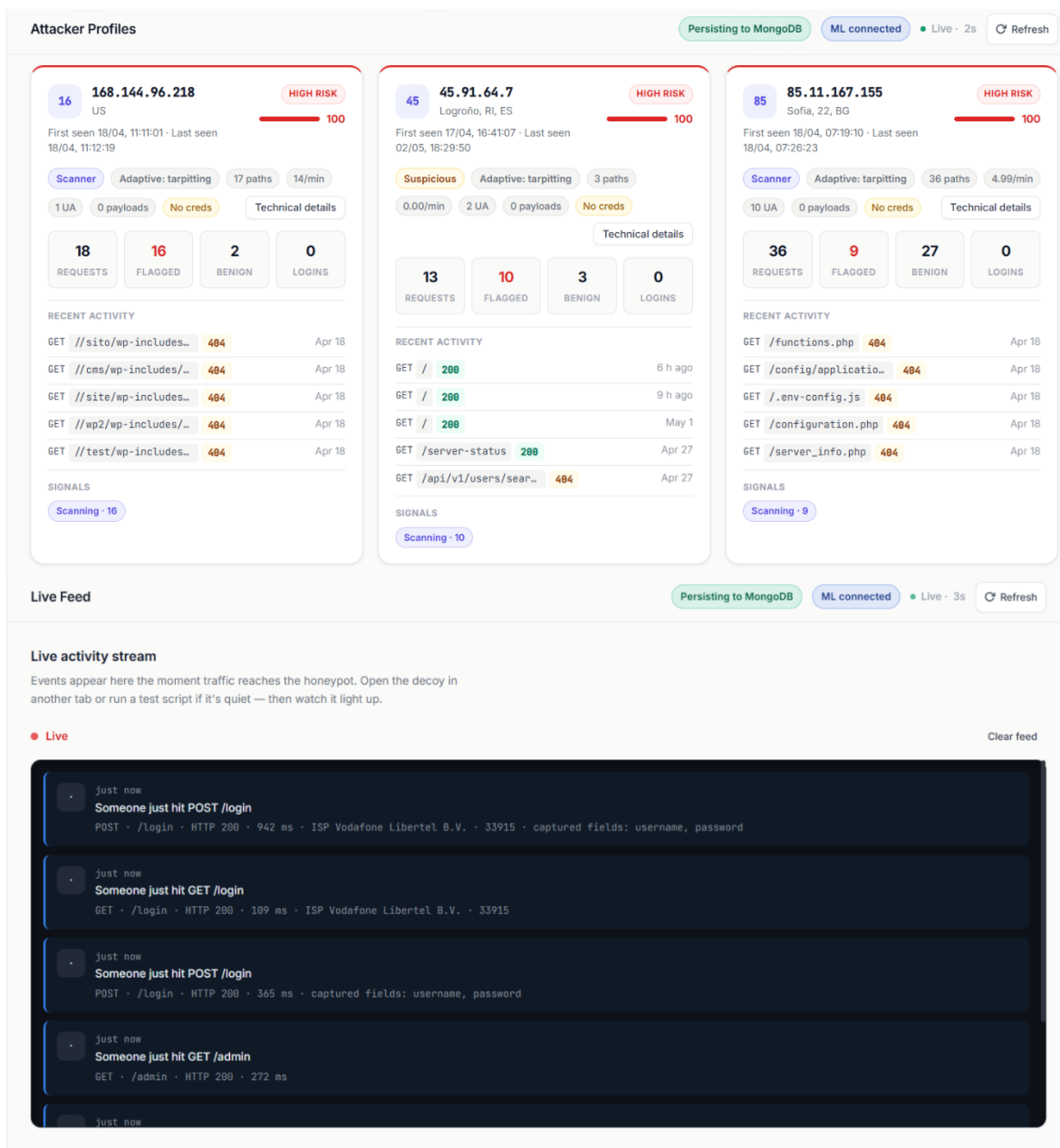


FIGURE 6. TRAFFIC ANALYSIS

5.4 Study Limitations

The honeypot is deployed for research, but attack logs are not kept for long. Data is deleted regularly, so there is no multi-month archive to analyze. Results in this thesis are based on recent sessions, simulator runs, and dashboard views taken before the next cleanup.

Model accuracy for the Python classifier was not measured. Testing focused on rule-based detection, IP profiles, and adaptive responses shown in the admin panel. Traffic included both simulator scripts and some live hits on the decoy, but that is still a small sample compared with busy public servers.

Because records are removed, screenshots and report numbers had to be collected right after a test. Events deleted earlier could not be used later. The admin dashboard also needs strong login settings and correct proxy configuration if the system stays online on a network.

The honeypot also runs on a research server, not only on a local PC. Logs are deleted often on purpose, so there is no big archive from many weeks of traffic. What appears in this chapter comes from simulator runs and from whatever was still visible on the dashboard before the next cleanup. Admin login and an IP allowlist protect the console; that is normal for a research setup, but it also means figures had to be saved while the data was still there.

5.5 Prototype Demonstration Report

The demo checked whether HoneypotAI behaves as one full system, not only as a fake website. A request should pass through the public route, logging, detection, adaptive logic, storage, dashboard API, and finally the admin screen. That chain was the main thing to verify. The split between / and /dashboard worked as planned. Anyone can open the decoy portal, while the console sits behind login. Suspicious paths such as /admin, /.env, or backup-style URLs showed up in the logs as probing or scanning rather than normal visits, which is what the honeypot is meant to do. Adaptive replies also showed during tests. Scanner-style traffic often got a normal decoy page, but heavier patterns could trigger delay, fake data, or extra fake routes. The admin dashboard made this visible through counters, charts, the live feed, and the map when geo data existed. If map points were missing, the other panels still loaded, which helped during quick test sessions. Before a new run, stored logs and profiles could be

cleared so the next simulation did not mix with the previous one. That made it easier to capture screenshots for brute force, SQL injection, and scanning as separate cases.

5.6 Dashboard and Deployment Observations

Most proof that the system worked came from the admin dashboard, not from extra tools.

After test:bruteforce, new lines appeared under Attacks with login-related types, and the same IP gained login attempt counts on Profiles. After test:sqli, /search requests with injection strings were marked malicious; some responses looked like fake query output in the simulator terminal. After test:scanner, one IP hit many paths in a short time and the profile leaned toward scanner once the path count crossed the threshold.

Overview counters moved up after each script. The Live feed showed events without refreshing the whole page. The map only drew dots when geo lookup worked; missing coords did not break the other panels.

The honeypot also ran on a live host for research. Dashboard access was limited by login and an IP allowlist. Attack logs are not kept for months; data is deleted regularly, so this chapter describes recent runs and simulator output, not a long traffic history. ML on port 5001 was optional; rule hits and dashboard logs were the main source for the results above.

During testing, the Overview page was the first place to look. Total events and category counts went up right after a simulator finished. That was enough to know the backend was counting traffic.

The Attacks tab listed each hit with IP, path, method, and attack type. Matching a simulator line to a log row was straightforward: a /login POST after test:bruteforce, a /search?q= line after test:sqli, many GETs to odd paths after test:scanner.

Profiles grouped behaviour by IP. One busy scanner IP often had many unique paths and a scanner-style label once the threshold was passed. Credentials submitted to fake login forms could also be viewed in the dashboard when testers typed into decoy forms during manual checks.

The Live feed updated without reloading the whole page. The map was hit-or-miss: some IPs got coordinates and a dot, others did not, especially on local tests. Missing map data did not stop the charts or the log table from loading, which mattered when capturing screenshots quickly before the next log wipe.

5.6.1 Admin Dashboard During Tests

Overview counters moved up right after each simulator. On Attacks, each run left rows that matched the script: POSTs to /login after brute force, /search?q= lines after SQLi, long lists of GET paths after the scanner.

Profiles tracked one IP at a time. A scanner run usually pushed path count up until the scanner threshold was reached. Submitted usernames and passwords from fake login forms also appeared when manual tests were done on the decoy.

The Live feed showed new events without reloading the page. The map only plotted IPs when geo lookup worked; on localhost or some remote hits there was no dot, but charts and logs still loaded. That mattered when grabbing screenshots before logs were wiped again.

5.7 Production Server Log Sample

HoneyPotAI was also run on a VPS under pm2 (honeypotai process, app path /var/www/html/honeypotAI). Output was read with pm2 logs honeypotai --lines 200. That sample is different from the local simulators: the IPs are public addresses from many countries, and the paths match what bots and scanners usually try on a live web host.

Most flagged events were scanning at medium severity. Examples from the log include /robots.txt, /sitemap.xml, /wp-admin/install.php, /.env, /.git/config, /owa/auth/logon.aspx, and /cgi-bin/luci/;stok=/locale. User agents such as zgrab/0.x and AhrefsBot/7.0 appear in the pattern list. Plain page views also show up as GET /, GET /login, GET /db, or GET /search with response times between roughly 50 ms and 1300 ms, which looks like normal browsing mixed with probes.

The adaptive layer is visible in the same lines. Sources marked suspicious often get delay (about 500–990 ms) or tarpit (about 5–9 seconds). After classification as scanner, the log shows fake_routes and follow-up GETs to random paths such

as /n13798hlcas8 or /cvkolv9te_0blfq1, which matches the dynamic decoy route behaviour in code.

Two problems appear in production but not in every local run. First, ML classification error: network timeout at `http://127.0.0.1:5001/classify` repeats often, so rule-based detection and profiling still handled traffic while the Python service was down or too slow. Second, ProfileManager logged a `CastError` for IPv6 address `2a06:98c0:3600::103` on field `capturedPayloads`, so profile updates for that source may be incomplete until the schema handling is fixed.

Security controls also show in the log. Lines like `Dashboard request blocked by IP allowlist for /favicon.ico` mean external scanners hit the honeypot host but were not allowed to reach dashboard assets. That is expected when `DASHBOARD_ALLOWED_IPS` is set on a public server.

This log excerpt supports the rest of the results chapter: the decoy receives real unsolicited traffic, classifies much of it as scanning, applies adaptive delay/tarpit/fake routes, and keeps the admin surface restricted. It is a snapshot only (200 lines, one session window), not a full traffic study, and logs are still rotated on the research host as noted in Section 5.4.

6. DISCUSSION AND CONCLUSIONS

This document mixes research background with a practical account of HoneypotAI, the adaptive honeypot developed for this thesis. The goal was to connect general ideas about threat detection and adaptive decoys with the details of a system that was actually built, tested, and deployed for research. Architecture, code structure, admin dashboard behavior, and test runs were described in depth so the thesis would not stay only at theory level.

HoneypotAI combines a fake public portal, rule-based detection, IP profiles, adaptive replies, optional machine learning, and a secured admin console. Local simulators and live traffic showed that requests are logged, classified, and visible on the dashboard. Adaptive actions such as delay and fake data appeared when attack patterns built up during runs.

Some limits remain. Logs are cleared regularly and are not kept for months. The ML module was not evaluated with formal accuracy scores. There was no long comparison against a static honeypot. Those points should be kept in mind when reading the results.

Future work could use real logs for ML training, link classifier output more closely to adaptive policy, and run longer tests with anonymised retention. Even with these limits, the project shows how an adaptive web honeypot can be implemented and reviewed in a research setting.

In closing, the reader will find research explanation mixed with detail from the author's own implementation work. The extra material was included to enrich the thesis and to make clear what was designed, what was built, and what was observed during testing.

7. REFERENCES

- [1] L. Spitzner, *Honeypots: Tracking Hackers*. Addison-Wesley, 2002.
- [2] N. Provos and T. Holz, *Virtual Honeypots: From Botnet Tracking to Intrusion Detection*. Addison-Wesley, 2007.
- [3] F. Cohen, "The Use of Deception Techniques: Honeypots and Decoys," in *Handbook of Information Security*, H. Bidgoli, Ed. John Wiley & Sons, 2006.
- [4] M. Bringer, C. Chelmecki, and H. Fujinoki, "A Survey: Recent Advances and Future Trends in Honeypot Research," *International Journal of Computer Network and Information Security*, vol. 4, no. 10, pp. 63–75, 2012.
- [5] A. Sommer and V. Paxson, "Outside the Closed World: On Using Machine Learning For Network Intrusion Detection," in *Proc. IEEE Symposium on Security and Privacy*, Oakland, CA, 2010.
- [6] M. A. Ferrag, L. Maglaras, A. Ahmim, M. Derdour, and H. Janicke, "RDTIDS: Rules and Decision Tree-Based Intrusion Detection System for Internet-of-Things Networks," *Future Internet*, vol. 12, no. 3, p. 44, 2020.
- [7] R. Chalapathy and S. Chawla, "Deep Learning for Anomaly Detection: A Survey," *arXiv:1901.03407*, 2019.
- [8] C. Yin, Y. Zhu, J. Fei, and X. He, "A Deep Learning Approach for Intrusion Detection Using Recurrent Neural Networks," *IEEE Access*, vol. 5, pp. 21954–21961, 2017.
- [9] T. Holz and F. Raynal, "Detecting Honeypots and Other Suspicious Environments," in *Proc. IEEE Workshop on Information Assurance and Security*, 2005.
- [10] M. Carroll and S. Grobler, "Adaptive Honeypot Strategy with Reinforcement Learning," in *Proc. International Conference on Cyber Warfare and Security*, 2021.
- [11] S. Khorshed, T. Wittemann, and P. Robinson, "Container-Based Honeypot Deployment for Dynamic Threat Intelligence," in *Proc. ACM CCS Workshop on Moving Target Defense*, 2020.

- [12] The HoneyNet Project, *Know Your Enemy: Learning About Security Threats*, 2nd ed. Addison-Wesley, 2004.
- [13] D. Fraunholz, D. Reti, S. D. Anton, and H. D. Schotten, "YAAS - On the Attribution of Honeypot Data," *International Journal of Cyber Situational Awareness*, vol. 2, no. 1, 2017.
- [14] M. Pa Pa, S. Suzuki, K. Yoshioka, T. Matsumoto, T. Kasama, and C. Rossow, "IoT POT: Analysing the Rise of IoT Compromises," in *Proc. USENIX Workshop on Offensive Technologies*, 2015.
- [15] CAIDA, "Network Telescope - UCSD-NT," Center for Applied Internet Data Analysis, 2023. [Online]. Available: https://www.caida.org/projects/network_telescope/
- [16] J. Bilge and T. Dumitras, "Before We Knew It: An Empirical Study of Zero-Day Attacks in the Real World," in *Proc. ACM CCS*, 2012.
- [17] Q. Zhu and T. Basar, "Game-Theoretic Approach to Feedback-Driven Multi-Stage Moving Target Defense," in *Proc. GameSec*, 2013.
- [18] T. Bhatt et al., "The T-Pot Multi-Honeypot Platform," Technical Report, Deutsche Telekom Security GmbH, 2021.
- [19] Mandiant, "M-Trends 2022: Special Report," Mandiant Inc., 2022.
- [20] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design Science in Information Systems Research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.